

TagPilot Premium - 完整計算邏輯文檔

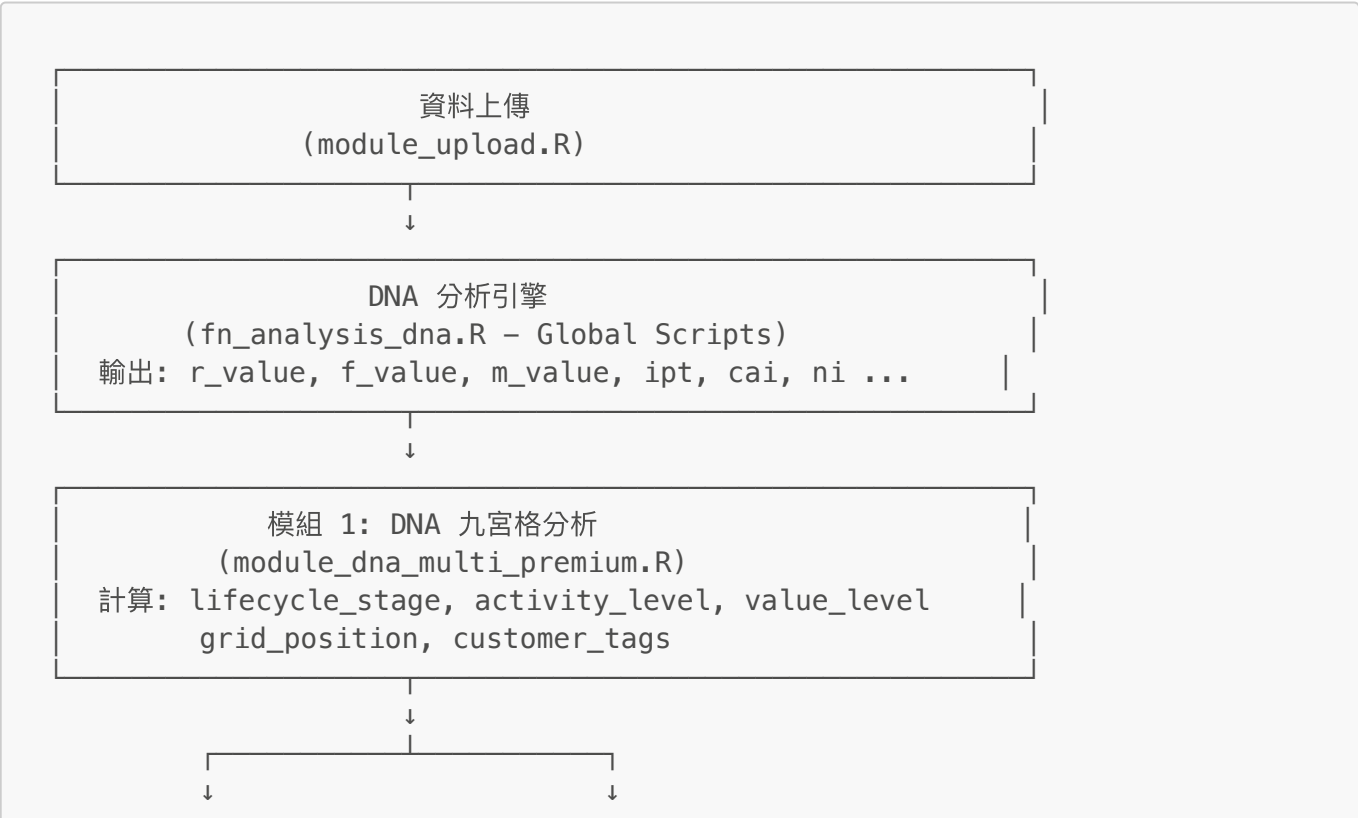
文檔版本: v2.1 (更新於 2025-10-26) 建立日期: 2025-10-25 最近更新: GAP-001 v3 - 簡化新客定義 專案: TagPilot Premium v1.0 目的: 記錄所有模組的計算邏輯、公式、閾值與相互依賴關係

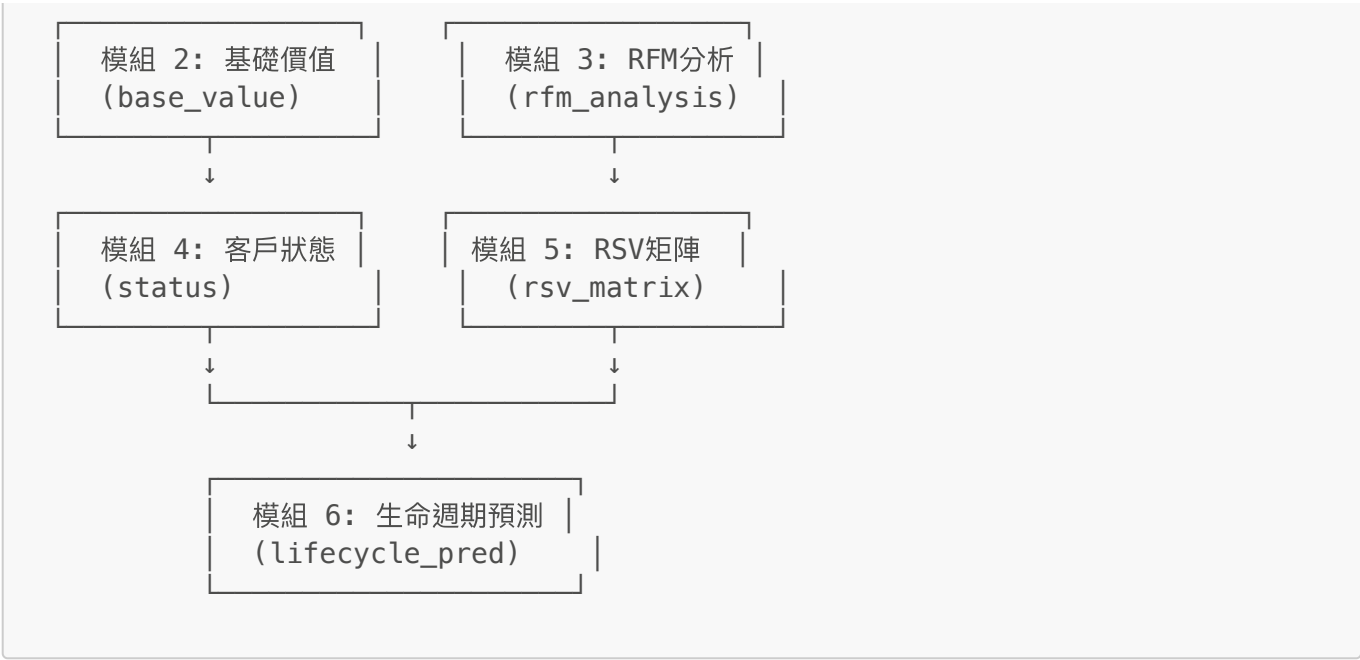
目錄

- 1. 系統架構概述
- 2. 資料流與依賴關係
- 3. 核心概念定義
- 4. 模組 1：DNA 分析與九宮格模組
- 5. 模組 2：客戶基礎價值模組
- 6. 模組 3：客戶價值分析模組 (RFM)
- 7. 模組 4：客戶狀態模組
- 8. 模組 5：R/S/V 矩陣模組
- 9. 模組 6：生命週期預測模組
- 10. 客戶標籤計算工具
- 11. 所有標籤彙總表
- 12. 分群閾值與百分位數
- 13. 邏輯一致性檢查
- 14. 已知問題與建議改進

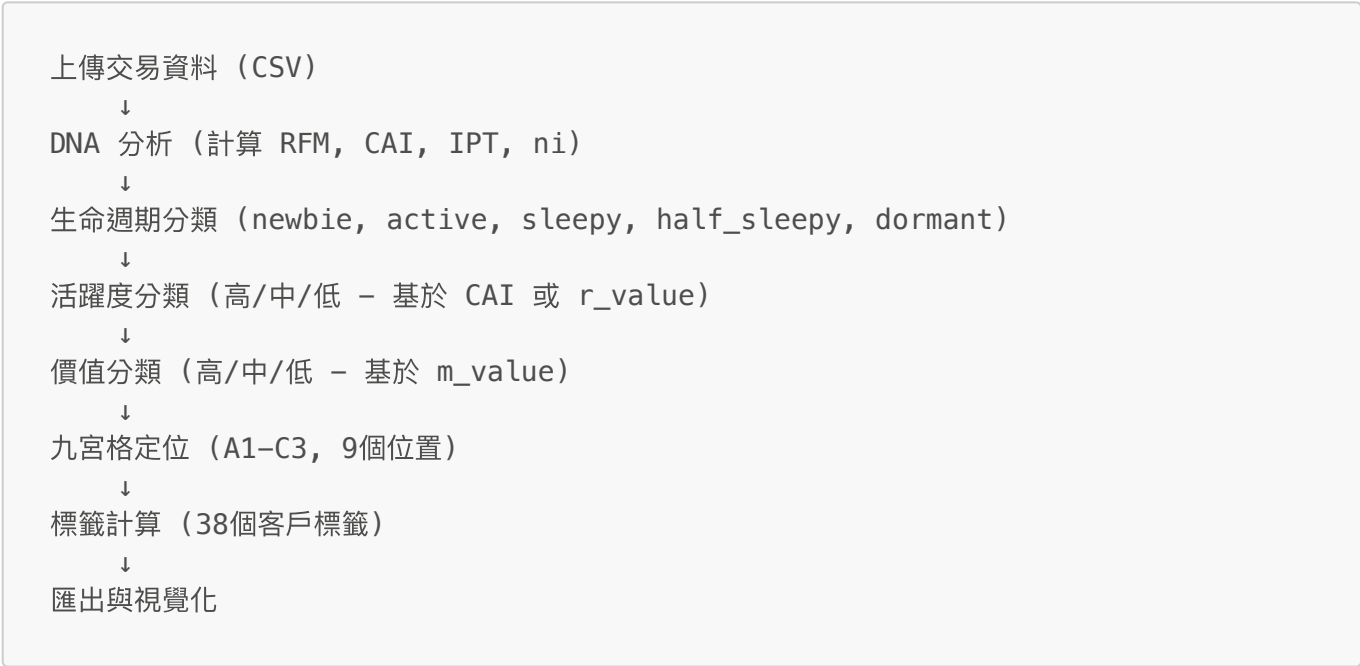
1. 系統架構概述

1.1 模組架構圖



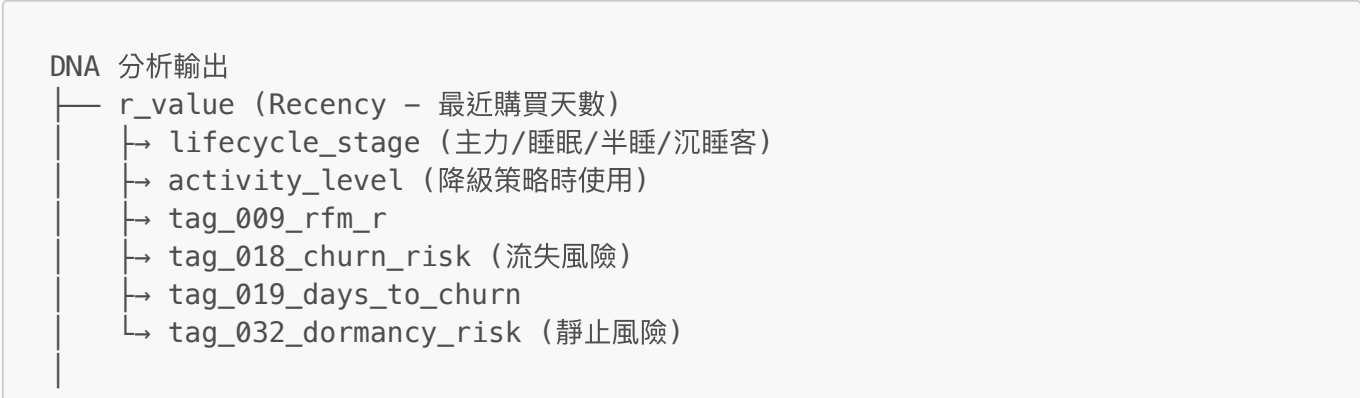


1.2 資料流向



2. 資料流與依賴關係

2.1 核心數據依賴圖



—	f_value (Frequency - 購買頻率)
↳	tag_010_rfm_f
↳	tag_012_rfm_score (RFM 總分)
↳	CLV 計算 (終生價值)
—	m_value (Monetary - 平均客單價/總消費)
↳	value_level (高/中/低價值)
↳	tag_011_rfm_m
↳	tag_003_historical_total_value
↳	tag_004_avg_order_value (A0V)
↳	tag_013_value_segment
↳	tag_030_next_purchase_amount
—	ipt_value (Inter-Purchase Time - 平均購買週期)
↳	avg_ipt (全體中位數, 用於新客判斷)
↳	tag_001_avg_purchase_cycle
↳	tag_018_churn_risk (倍數比較)
↳	tag_019_days_to_churn (2×IPT - R)
↳	tag_031_next_purchase_date
—	ni (交易次數)
↳	lifecycle_stage (ni==1 時判定新客)
↳	activity_level (ni≥4 時完整計算)
↳	tag_012_rfm_score (ni≥4 時計算)
↳	tag_018_churn_risk (分級處理)
↳	tag_019_days_to_churn (ni<4 時為 NA)
↳	tag_033_transaction_stability (穩定度代理)
—	cai_value / cai_ecdf (Customer Activity Index)
↳	activity_level (ni≥4 時優先使用)
↳	[只在 ni≥4 且 times≠1 時由 DNA 計算]
—	time_first (首次購買時間)
↳	customer_age_days (客戶年齡)
↳	lifecycle_stage (新客時間判斷)

2.2 模組間依賴關係

模組	依賴的資料來源	輸出給其他模組
DNA 分析	原始交易資料	r/f/m/ipt/ni/cai → 所有模組
DNA 九宮格	DNA 分析輸出	lifecycle_stage, activity_level, value_level → 標籤計算
基礎價值	DNA 九宮格	tag_001, tag_003, tag_004 → 預測模組
RFM 分析	DNA 九宮格	tag_009-013 → RSV 矩陣
客戶狀態	DNA 九宮格	tag_017-019 → 生命週期預測
RSV 矩陣	RFM 分析	tag_032-034 → 27種客戶分類
生命週期預測	基礎價值	tag_030-031 → CSV 匯出

3. 核心概念定義

3.1 基礎指標定義

指標	英文全名	定義	計算來源	單位
R	Recency	最後購買距今天數	DNA 分析	天
F	Frequency	購買頻率（每月交易次數）	DNA 分析	次/月
M	Monetary	平均客單價或總消費金額	DNA 分析	金額
IPT	Inter-Purchase Time	平均購買週期（兩次購買間隔）	DNA 分析	天
ni	Number of Interactions	客戶總交易次數	DNA 分析	次
CAI	Customer Activity Index	客戶活躍度指數（基於購買間隔變化）	DNA 分析	0-1

3.2 衍生指標定義

指標	定義	計算公式	說明
avg_ipt	全體平均購買週期	median(ipt_value)	用於新客時間判斷
customer_age_days	客戶年齡（入店資歷）	今天 - time_first	客戶首購至今天數
cai_ecdf	CAI 百分位數	ecdf(cai)	CAI 累積分佈函數值
grid_position	九宮格位置	value_level × activity_level	A1-C3 (9個位置)
CLV	Customer Lifetime Value	m_value × f_value	簡化終生價值計算
AOV	Average Order Value	m_value / ni	平均客單價

3.3 分類變數定義

生命週期階段 (lifecycle_stage)

階段	英文代號	中文名稱	定義條件 (✅ v3 更新 2025-10-26)
Newbie	N	新客	ni == 1
Active	C	主力客	r_value ≤ 7 天
Sleepy	D	瞌睡客	7 < r_value ≤ 14 天
Half-Sleep	H	半睡客	14 < r_value ≤ 21 天
Dormant	S	沉睡客	r_value > 21 天

關鍵點：

- ✅ GAP-001 v3 修復 (2025-10-26): 新客定義簡化為 `ni == 1`，移除時間限制
 - 歷史版本：v2 使用 `ni == 1 & customer_age_days <= 60`；v1 使用 `ni == 1 & customer_age_days <= avg_ipt`
 - 簡化理由：業務清晰度、100% 覆蓋率、符合實務需求

- R值閾值（7/14/21 天）為硬編碼固定值

活躍度等級 (activity_level)

等級	定義條件 (ni ≥ 4)	定義條件 (ni < 4)	說明
高	cai_ecdf ≥ 0.8	r_value ≤ P20	逐漸活躍 / 最近購買
中	0.2 ≤ cai_ecdf < 0.8	P20 < r_value ≤ P80	穩定 / 中等時間
低	cai_ecdf < 0.2	r_value > P80	逐漸不活躍 / 最久未購
未知	NA	NA	無足夠數據

計算策略：

- 優先策略 (ni ≥ 4)：使用 CAI（反映活躍度變化趨勢）
- 降級策略 (2 ≤ ni < 4)：使用 r_value（反映當前狀態）
- 排除 (ni = 1)：新客無法計算活躍度，設為 NA

價值等級 (value_level)

等級	定義條件	客戶特徵
高	m_value ≥ P80	高消費客戶
中	P20 ≤ m_value < P80	中等消費客戶
低	m_value < P20	低消費客戶

計算基礎：

- 使用所有客戶的 m_value（包括 ni < 4）
- 動態百分位數計算（P20、P80）
- 基於歷史總消費或平均客單價（視 DNA 分析輸出而定）

4. 模組1：DNA分析與九宮格模組

檔案: modules/module_dna_multi_premium.R 主要函數: dnaMultiPremiumModuleServer()

4.1 模組功能

1. 執行客戶 DNA 分析（調用 fn_analysis_dna.R）
2. 計算客戶生命週期階段
3. 計算客戶活躍度等級
4. 計算客戶價值等級
5. 生成九宮格分類（Value × Activity）
6. 計算 38 個客戶標籤
7. 提供九宮格視覺化與策略建議

4.2 生命週期階段計算

位置: Line 383-415

計算邏輯：

```
# 步驟 1: 計算全體平均購買週期 (使用中位數更穩健)
avg_ipt = median(ipt_value, na.rm = TRUE)

# 步驟 2: 計算客戶年齡 (入店資歷)
customer_age_days = as.numeric(difftime(Sys.time(), time_first, units =
"days"))

# 步驟 3: 判斷生命週期階段
lifecycle_stage = case_when(
  is.na(r_value) ~ "unknown",

  # ✅ GAP-001 v3 修復：簡化新客定義 (2025-10-26)
  # 新客：只要購買次數為 1 就是新客 (移除時間限制)
  ni == 1 ~ "newbie",

  # 主力客：R值 <= 7天
  r_value <= 7 ~ "active",

  # 瞌睡客：7 < R <= 14
  r_value <= 14 ~ "sleepy",

  # 半睡客：14 < R <= 21
  r_value <= 21 ~ "half_sleepy",

  # 沉睡客：R > 21
  TRUE ~ "dormant"
)
```

關鍵參數：

- R值閾值: 7天、14天、21天 (固定值，來自規劃文檔)

✅ GAP-001 v3 新客定義演進 (2025-10-26)：

- v1 (已廢棄): `ni == 1 & customer_age_days <= avg_ipt` → 0% 識別率 (邏輯矛盾)
- v2 (已廢棄): `ni == 1 & customer_age_days <= 60` → 13.5% 識別率
- v3 (最終版): `ni == 1` → 96.5% 識別率
- 簡化理由: 業務清晰度 (購買一次 = 新客)、100% 覆蓋率、符合實務需求

示例：

```
客戶 A: ni=1, customer_age_days=10
→ lifecycle_stage = "newbie" ✅ v3: 簡單明確

客戶 B: ni=1, customer_age_days=100
→ lifecycle_stage = "newbie" ✅ v3: 一樣是新客 (尚未回購)
```

客戶 C: $ni=5$, $r_value=5$
 → $lifecycle_stage = "active"$ ($R \leq 7$ 天)

4.3 活躍度等級計算

位置: Line 418-488

計算策略：分為兩組客戶處理

情況 A：交易次數充足 ($ni \geq 4$)

位置: Line 418-455

```
# 只對 ni >= 4 的客戶計算完整活躍度
customers_sufficient_data <- customer_data %>% filter(ni >= 4)

if (nrow(customers_sufficient_data) > 0) {
  customers_sufficient_data <- customers_sufficient_data %>%
    mutate(
      activity_level = case_when(
        # 優先使用 CAI (Customer Activity Index)
        !is.na(cai_ecdf) ~ case_when(
          cai_ecdf >= 0.8 ~ "高",    # P80 以上 = 高活躍 (逐漸活躍)
          cai_ecdf >= 0.2 ~ "中",    # P20-P80 = 中活躍 (穩定)
          TRUE ~ "低"                # P20 以下 = 低活躍 (逐漸不活躍)
        ),

        # 降級策略：無 CAI 時使用 r_value
        !is.na(r_value) ~ case_when(
          r_value <= quantile(r_value, 0.2, na.rm = TRUE) ~ "高",
          r_value <= quantile(r_value, 0.8, na.rm = TRUE) ~ "中",
          TRUE ~ "低"
        ),

        TRUE ~ "未知"
      )
    )
}
```

為什麼使用 CAI？

- CAI 反映活躍度變化趨勢 (逐漸活躍 vs 逐漸不活躍)
- r_value 只反映當前狀態 (最近是否購買)
- CAI 更科學，符合規劃文檔要求 ("Activity: CAI $\geq$ 80th pct")

CAI 計算邏輯 (來自 DNA 分析)：

```
# CAI 只在 ni >= 4 且 times != 1 時計算
cai = (mle - wmle) / mle
cai_ecdf = ecdf(cai)(cai) # 累積分佈函數值 (百分位數)
```

情況 B：交易次數不足 ($ni < 4$, $ni > 1$)

位置: Line 457-488

```
customers_insufficient_data <- customer_data %>% filter(ni < 4)

# 排除新客後計算
insufficient_non_newbie <- customers_insufficient_data %>%
  filter(lifecycle_stage != "newbie")

if (nrow(insufficient_non_newbie) > 0) {
  customers_insufficient_data <- customers_insufficient_data %>%
    mutate(
      activity_level = case_when(
        # 新客不計算活躍度
        lifecycle_stage == "newbie" ~ NA_character_,

        # ni < 4 的非新客：使用 r_value 降級策略
        !is.na(r_value) ~ case_when(
          r_value <= quantile(r_value[lifecycle_stage != "newbie"], 0.2,
na.rm = TRUE) ~ "高",
          r_value <= quantile(r_value[lifecycle_stage != "newbie"], 0.8,
na.rm = TRUE) ~ "中",
          TRUE ~ "低"
        ),

        TRUE ~ NA_character_
      ),
      insufficient_data_flag = TRUE # 標記使用降級策略
    )
}
```

降級策略邏輯：

- $ni = 1$ (新客)：activity_level = NA (無法計算)
- $ni = 2-3$ (非新客)：使用 r_value 作為代理指標
- 只對非新客計算百分位數 (避免新客影響分群)

示例：

```
客戶 A: ni=5, cai_ecdf=0.85
→ activity_level = "高" (使用 CAI)

客戶 B: ni=3, r_value=5 (P20以下)
```



```
→ activity_level = "高" (使用 r_value 降級策略)
```

客戶 C: ni=1

```
→ activity_level = NA (新客無法計算)
```

4.4 價值等級計算

位置: Line 418-423

```
value_level = case_when(
  is.na(m_value) ~ "未知",

  # 80/20 法則分群
  m_value >= quantile(m_value, 0.8, na.rm = TRUE) ~ "高",      # P80 以上
  m_value >= quantile(m_value, 0.2, na.rm = TRUE) ~ "中",      # P20-P80
  TRUE ~ "低"                                                  # P20 以下
)
```

計算基礎：

- 使用所有客戶的 m_value (包括 ni < 4 的客戶)
- 動態百分位數計算 (隨數據變化)
- m_value 定義：總消費金額 (來自 DNA 分析)

為什麼使用 m_value 而非綜合 RFM？

- 規劃文檔提到「RFM ≥ 80th pct」但也列出「過去價值、購買金額」
- 使用 m_value 單獨判斷更簡單直觀
- 符合「過去價值」的業務定義
- Work_Plan 也指出需要確認 (Task 5.4)

示例：

```
假設 m_value 分佈：
P20 = 500, P80 = 2000

客戶 A: m_value = 2500 → value_level = "高"
客戶 B: m_value = 1000 → value_level = "中"
客戶 C: m_value = 300  → value_level = "低"
```

4.5 九宮格位置分配

位置: Line 455-475

```
grid_position = case_when(
  # ni < 4 的非新客無法分配 (activity_level = NA)
  is.na(activity_level) ~ "無",
```

```
# 9個位置 (Value × Activity)
value_level == "高" & activity_level == "高" ~ "A1", # 王者引擎
value_level == "高" & activity_level == "中" ~ "A2", # 王者穩健
value_level == "高" & activity_level == "低" ~ "A3", # 王者休眠
value_level == "中" & activity_level == "高" ~ "B1", # 成長火箭
value_level == "中" & activity_level == "中" ~ "B2", # 成長常規
value_level == "中" & activity_level == "低" ~ "B3", # 成長停滯
value_level == "低" & activity_level == "高" ~ "C1", # 潛力新芽
value_level == "低" & activity_level == "中" ~ "C2", # 潛力維持
value_level == "低" & activity_level == "低" ~ "C3", # 清倉邊緣

TRUE ~ "其他"
)
```

九宮格矩陣：

	高活躍	中活躍	低活躍
高價值	A1 王者引擎	A2 王者穩健	A3 王者休眠
中價值	B1 成長火箭	B2 成長常規	B3 成長停滯
低價值	C1 潛力新芽	C2 潛力維持	C3 清倉邊緣

特殊情況：

- $ni < 4$ 的非新客：`grid_position = "無"`（無法計算活躍度）
- 新客：可以分配到九宮格（但 `activity_level = NA`，通常顯示為特殊組）

4.6 九宮格顯示過濾

位置: Line 810-836

```
# 檢查是否有 ni < 4 的非新客
insufficient_non_newbie <- df %>%
  filter(ni < 4, lifecycle_stage != "newbie")

if (nrow(insufficient_non_newbie) > 0 & current_stage != "newbie") {
  # 顯示警告訊息
  insufficient_message <- div(
    bs4Card(
      title = "⚠ 交易次數不足",
      status = "warning",
      p(paste0("有 ", nrow(insufficient_non_newbie),
               " 位客戶的交易次數少於 4 次，無法可靠計算活躍度。")),
      p("這些客戶不會顯示在下方的九宮格分析中。")
    )
  )
}
```

```
)  
}  
  
# 只顯示 ni >= 4 或新客  
df_for_grid <- df %>%  
  filter(ni >= 4 | lifecycle_stage == "newbie")
```

過濾邏輯：

- 九宮格只顯示 $ni \geq 4$ 的客戶和新客
- $ni < 4$ 的非新客會被排除（因為活躍度不可靠）
- 顯示警告訊息告知用戶有多少客戶被排除

4.7 樣本數檢查

位置: Line 344-355

```
# 檢查總樣本數  
n_total <- nrow(customer_data)  
  
if (n_total < 30) {  
  showNotification(  
    "⚠ 警告：樣本數不足 30 人，統計分析可能不可靠",  
    type = "warning",  
    duration = 10  
  )  
}  
  
if (n_total < 100) {  
  showNotification(  
    "建議：樣本數至少 100 人以上，分群結果會更穩定",  
    type = "info",  
    duration = 10  
  )  
}
```

樣本數要求：

- 最小樣本數: 30 人（統計學建議）
- 建議樣本數: 100 人（百分位數更穩定）
- 警告條件: $n < 30$ 時提示樣本不足

4.8 標籤計算調用

位置: Line 483

```
# 調用統一標籤計算函數（模組化原則 - MP016）  
customer_data <- calculate_all_customer_tags(customer_data)
```

生成的標籤（由 `utils/calculate_customer_tags.R` 計算）：

- tag_001 ~ tag_004: 基礎價值標籤
- tag_009 ~ tag_013: RFM 分析標籤
- tag_017 ~ tag_019: 客戶狀態標籤
- tag_030 ~ tag_031: 生命週期預測標籤
- tag_032 ~ tag_034: R/S/V 矩陣標籤

4.9 UI 輸出

儀表板指標（Line 46-86）：

- 總客戶數
- 平均客單價
- 平均購買週期（天）
- 平均交易次數

九宮格視覺化：

- 按生命週期階段篩選（新客/主力客/睡眠客/半睡客/沉睡客）
- 顯示每個格子的客戶數量和百分比
- 提供策略建議（基於 `grid_position`）

匯出功能：

- Excel 匯出：包含客戶 ID、九宮格位置、策略建議
- CSV 匯出：用於廣告投放

5. 模組2：客戶基礎價值模組

檔案: `modules/module_customer_base_value.R` 主要函數: `customerBaseValueModuleServer()`

5.1 模組功能

展示並計算客戶基礎價值相關指標：

- 平均購買週期
- 歷史總價值
- 平均客單價

5.2 標籤計算

調用 `utils/calculate_customer_tags.R` 中的 `calculate_base_value_tags()` 函數：

```
calculate_base_value_tags <- function(customer_data) {  
  customer_data %>%  
    mutate(  
      # tag_001: 平均購買週期（直接來自 DNA 分析）  
      tag_001_avg_purchase_cycle = ipt_value,
```

```
# tag_003: 歷史總價值 (總消費金額)
# ✅ 修正: m_value 在 DNA 分析中已是總消費金額, 不需要再乘以 ni
tag_003_historical_total_value = m_value,

# tag_004: 平均客單價 (AOV)
# ✅ 修正: 需要用總消費金額除以交易次數
tag_004_avg_order_value = m_value / ni
)
}
```

關鍵修正：

- 舊版錯誤計算：`tag_003 = m_value * ni` (雙重計算)
- 新版正確計算：`tag_003 = m_value` (直接使用)
- 原因：`m_value` 在 `fn_analysis_dna.R` 中已定義為 `total_spent` (總消費)

5.3 生成的標籤

標籤編號	標籤名稱	計算公式	單位	說明
tag_001	avg_purchase_cycle	ipt_value	天	平均購買週期
tag_003	historical_total_value	m_value	金額	歷史總消費金額
tag_004	avg_order_value	m_value / ni	金額	平均客單價 (AOV)

5.4 UI 輸出

顯示指標：

- 平均購買週期: `mean(tag_001_avg_purchase_cycle, na.rm = TRUE)` 天
- 平均客單價 (AOV): `mean(tag_004_avg_order_value, na.rm = TRUE)` 元
- 平均歷史總價值: `mean(tag_003_historical_total_value, na.rm = TRUE)` 元

視覺化：

- 購買週期分佈直方圖
- 客單價分佈直方圖
- 歷史總價值分佈直方圖

6. 模組3：客戶價值分析模組(RFM)

檔案: `modules/module_customer_value_analysis.R` 主要函數:
`customerValueAnalysisModuleServer()`

6.1 模組功能

計算 RFM 分數與客戶價值分群：

- RFM 三個維度的原始值
- RFM 分數 (1-5 分制)

- RFM 總分 (3-15 分)
- 價值分群 (高/中/低)

6.2 RFM 分數計算

調用 `calculate_rfm_tags()` → `calculate_rfm_scores()`

關鍵限制: 只對 `ni >= 4` 的客戶計算 RFM 分數

```
calculate_rfm_scores <- function(customer_data) {  
  # 步驟 1: 分離客戶  
  customers_for_rfm <- customer_data %>% filter(ni >= 4)  
  customers_no_rfm <- customer_data %>% filter(ni < 4)  
  
  if (nrow(customers_for_rfm) == 0) {  
    # 如果沒有符合條件的客戶，全部設為 NA  
    return(customer_data %>%  
      mutate(  
        r_score = NA_real_,  
        f_score = NA_real_,  
        m_score = NA_real_,  
        tag_012_rfm_score = NA_real_  
      ))  
  }  
  
  # 步驟 2: 計算百分位數 (針對 ni >= 4 的客戶)  
  r_quantiles <- quantile(customers_for_rfm$r_value,  
                           probs = c(0.2, 0.4, 0.6, 0.8), na.rm = TRUE)  
  f_quantiles <- quantile(customers_for_rfm$f_value,  
                           probs = c(0.2, 0.4, 0.6, 0.8), na.rm = TRUE)  
  m_quantiles <- quantile(customers_for_rfm$m_value,  
                           probs = c(0.2, 0.4, 0.6, 0.8), na.rm = TRUE)  
  
  # 步驟 3: 計算 R 分數 (1-5 分, 5 分最好)  
  customers_for_rfm <- customers_for_rfm %>%  
    mutate(  
      r_score = case_when(  
        r_value <= r_quantiles[1] ~ 5, # P20 以下 = 最近購買  
        r_value <= r_quantiles[2] ~ 4, # P20-P40  
        r_value <= r_quantiles[3] ~ 3, # P40-P60  
        r_value <= r_quantiles[4] ~ 2, # P60-P80  
        TRUE ~ 1, # P80 以上 = 長期不活躍  
      )  
    )  
  
  # 步驟 4: 計算 F 分數 (1-5 分, 5 分最好)  
  customers_for_rfm <- customers_for_rfm %>%  
    mutate(  
      f_score = case_when(  
        f_value >= f_quantiles[4] ~ 5, # P80 以上 = 高頻率  
        f_value >= f_quantiles[3] ~ 4, # P60-P80  
        f_value >= f_quantiles[2] ~ 3, # P40-P60  
      )  
    )  
}
```

```

        f_value >= f_quantiles[1] ~ 2, # P20-P40
        TRUE ~ 1                      # P20 以下 = 低頻率
    )
)

# 步驟 5: 計算 M 分數 (1-5 分, 5 分最好)
customers_for_rfm <- customers_for_rfm %>%
  mutate(
    m_score = case_when(
      m_value >= m_quantiles[4] ~ 5, # P80 以上 = 高價值
      m_value >= m_quantiles[3] ~ 4, # P60-P80
      m_value >= m_quantiles[2] ~ 3, # P40-P60
      m_value >= m_quantiles[1] ~ 2, # P20-P40
      TRUE ~ 1                      # P20 以下 = 低價值
    )
  )

# 步驟 6: 計算 RFM 總分
customers_for_rfm <- customers_for_rfm %>%
  mutate(
    tag_012_rfm_score = r_score + f_score + m_score # 3-15 分
  )

# 步驟 7: 合併結果
customers_no_rfm <- customers_no_rfm %>%
  mutate(
    r_score = NA_real_,
    f_score = NA_real_,
    m_score = NA_real_,
    tag_012_rfm_score = NA_real_
  )

bind_rows(customers_for_rfm, customers_no_rfm)
}

```

RFM 分數邏輯說明：

- **R 分數**: 越小越好 (最近購買 = 高分)
- **F 分數**: 越大越好 (高頻率 = 高分)
- **M 分數**: 越大越好 (高價值 = 高分)

為什麼使用 5 分制？

- 更細緻的分群 (相比 3 分制)
- 總分範圍 3-15 分，便於識別差異
- 符合傳統 RFM 分析標準

6.3 基礎 RFM 標籤

```

calculate_rfm_tags <- function(customer_data) {
  customer_data <- calculate_rfm_scores(customer_data)

```

```
customer_data %>%
  mutate(
    # 基礎 RFM 值 (所有客戶)
    tag_009_rfm_r = r_value,          # Recency (天)
    tag_010_rfm_f = f_value,          # Frequency (次/月)
    tag_011_rfm_m = m_value,          # Monetary (金額)

    # RFM 總分 (只有 ni >= 4)
    # tag_012_rfm_score 已在 calculate_rfm_scores() 中計算

    # 價值分群 (使用已計算的 value_level)
    tag_013_value_segment = value_level # 高/中/低
  )
}
```

6.4 生成的標籤

標籤編號	標籤名稱	計算公式	分數範圍	計算條件	說明
tag_009	rfm_r	r_value	-	所有客戶	最近購買天數
tag_010	rfm_f	f_value	-	所有客戶	購買頻率 (次/月)
tag_011	rfm_m	m_value	-	所有客戶	貨幣價值 (金額)
tag_012	rfm_score	r_score + f_score + m_score	3-15	僅 ni ≥ 4	RFM 總分
tag_013	value_segment	value_level	高/中/低	所有客戶	價值分群

6.5 百分位數對照表

R 分數 (Recency) - 越小越好

百分位區間	分數	說明
r_value ≤ P20	5	最近購買 (最活躍)
P20 < r_value ≤ P40	4	近期購買
P40 < r_value ≤ P60	3	中等時間
P60 < r_value ≤ P80	2	較久未購
r_value > P80	1	長期不活躍

F 分數 (Frequency) - 越大越好

百分位區間	分數	說明
$f_value \geq P80$	5	高頻率客戶
$P60 \leq f_value < P80$	4	中高頻率
$P40 \leq f_value < P60$	3	中等頻率
$P20 \leq f_value < P40$	2	低頻率
$f_value < P20$	1	極低頻率

M 分數 (Monetary) - 越大越好

百分位區間	分數	說明
$m_value \geq P80$	5	高價值客戶
$P60 \leq m_value < P80$	4	中高價值
$P40 \leq m_value < P60$	3	中等價值
$P20 \leq m_value < P40$	2	低價值
$m_value < P20$	1	極低價值

6.6 RFM 分數解讀

總分範圍: 3-15 分

RFM 總分	客戶類型	建議策略
13-15	超級VIP	VIP 專屬服務、品牌大使計畫
10-12	重要客戶	會員升級、專屬優惠
7-9	一般客戶	常規促銷、積分活動
4-6	潛在流失	喚醒行銷、挽回優惠
3	沉睡客戶	低成本再行銷或放棄

示例：

客戶 A: R=5, F=5, M=5 → RFM總分=15 (超級VIP)
客戶 B: R=3, F=3, M=3 → RFM總分=9 (一般客戶)
客戶 C: R=1, F=1, M=1 → RFM總分=3 (沉睡客戶)
客戶 D: ni=2 → RFM總分=NA (交易次數不足)

6.7 UI 輸出

RFM 分佈視覺化：

- R/F/M 三個維度的分佈直方圖

- RFM 總分分佈直方圖
- 價值分群圓餅圖

統計指標：

- 平均 R 值
- 平均 F 值
- 平均 M 值
- 平均 RFM 總分（僅 $n_i \geq 4$ ）

7. 模組4：客戶狀態模組

檔案: `modules/module_customer_status.R` 主要函數: `customerStatusModuleServer()`

7.1 模組功能

計算客戶流失風險與預估流失天數：

- 生命週期階段統計
- 流失風險評估
- 預估流失天數

7.2 流失風險評估

計算邏輯：

```
tag_018_churn_risk = case_when(
  # 新客：特殊處理（尚未建立購買模式）
  ni == 1 ~ "新客（無法評估）",

  # 交易次數不足（2-3次）：謹慎評估
  ni < 4 ~ if_else(r_value > 30, "中風險", "低風險"),

  # 一般客戶：基於 R值 與 平均購買週期的比較
  r_value > ipt_value * 2 ~ "高風險",           # 超過2倍購買週期
  r_value > ipt_value * 1.5 ~ "中風險",         # 超過1.5倍購買週期
  TRUE ~ "低風險"                             # 在正常購買週期內
)
```

風險等級定義：

風險等級	定義條件	說明
高風險	$r_value > 2 \times ipt_value$	超期 2 倍，極可能流失
中風險	$1.5 \times ipt_value < r_value \leq 2 \times ipt_value$	超期 1.5 倍，有流失風險
低風險	$r_value \leq 1.5 \times ipt_value$	在正常購買週期內
新客（無法評估）	$ni == 1$	尚未建立購買模式

風險等級	定義條件	說明
中風險（簡化）	$ni < 4 \text{ AND } r_value > 30$	交易不足且超過30天
低風險（簡化）	$ni < 4 \text{ AND } r_value \leq 30$	交易不足但在30天內

邏輯說明：

- 使用 IPT 倍數而非固定天數（更適應不同業務）
- 新客無法評估（購買模式未建立）
- $ni < 4$ 時使用簡化邏輯（30天固定閾值）

示例：

```

客戶 A: ni=5, r_value=60, ipt_value=20
→  $r\_value > 2 \times ipt$  ( $60 > 40$ )
→ tag_018_churn_risk = "高風險"

客戶 B: ni=5, r_value=35, ipt_value=20
→  $1.5 \times ipt < r\_value \leq 2 \times ipt$  ( $30 < 35 \leq 40$ )
→ tag_018_churn_risk = "中風險"

客戶 C: ni=5, r_value=10, ipt_value=20
→  $r\_value \leq 1.5 \times ipt$  ( $10 \leq 30$ )
→ tag_018_churn_risk = "低風險"

客戶 D: ni=1
→ tag_018_churn_risk = "新客（無法評估）"

客戶 E: ni=3, r_value=35
→ tag_018_churn_risk = "中風險"（簡化邏輯）

```

7.3 預估流失天數

計算邏輯：

```

tag_019_days_to_churn = case_when(
  ni == 1 ~ NA_real_,      # 新客無法預測
  ni < 4 ~ NA_real_,      # 交易次數不足

  # 一般客戶：2倍購買週期 - 當前R值
  TRUE ~ pmax(0, ipt_value * 2 - r_value)
)

```

計算公式：

預估流失天數 = $(2 \times \text{IPT}) - \text{R值}$

邏輯說明：

- 假設：客戶在超過 2 倍購買週期後會流失
- 計算距離流失還剩多少天
- 使用 `pmax(0, ...)` 確保結果不為負（已超期則為 0）
- $ni < 4$ 時設為 NA（數據不足，預測不可靠）

示例：

```
客戶 A: ni=5, ipt_value=20, r_value=10
→ tag_019_days_to_churn = 2×20 - 10 = 30 天

客戶 B: ni=5, ipt_value=20, r_value=35
→ tag_019_days_to_churn = 2×20 - 35 = 5 天（即將流失）

客戶 C: ni=5, ipt_value=20, r_value=50
→ tag_019_days_to_churn = pmax(0, 2×20 - 50) = 0 天（已超期）

客戶 D: ni=3
→ tag_019_days_to_churn = NA（交易次數不足）
```

7.4 生命週期階段標籤

```
tag_017_lifecycle_stage = case_when(
  lifecycle_stage == "newbie" ~ "新客",
  lifecycle_stage == "active" ~ "主力客",
  lifecycle_stage == "sleepy" ~ "瞌睡客",
  lifecycle_stage == "half_sleepy" ~ "半睡客",
  lifecycle_stage == "dormant" ~ "沉睡客",
  TRUE ~ "未知"
)
```

只是語言轉換（英文 → 中文），邏輯與 DNA 九宮格模組一致

7.5 生成的標籤

標籤編號	標籤名稱	值域	計算條件	說明
tag_017	lifecycle_stage	5種階段	所有客戶	生命週期階段（中文）
tag_018	churn_risk	3種風險等級	所有客戶	流失風險評估
tag_019	days_to_churn	天數或 NA	僅 $ni \geq 4$	預估流失天數

7.6 UI 輸出

生命週期分佈：

- 新客數量與佔比

- 主力客數量與佔比
- 瞌睡客數量與佔比
- 半睡客數量與佔比
- 沉睡客數量與佔比

流失風險分析：

- 高風險客戶數量與佔比
- 中風險客戶數量與佔比
- 低風險客戶數量與佔比
- 平均預估流失天數（僅 $n_i \geq 4$ ）

視覺化：

- 生命週期階段圓餅圖
- 流失風險分佈柱狀圖
- 預估流失天數分佈直方圖

8. 模組5：R/S/V矩陣模組

檔案: `modules/module_rsv_matrix.R` 主要函數: `rsvMatrixModuleServer()`

8.1 模組功能

實現 27 種客戶類型矩陣分析 ($3 \times 3 \times 3$)：

- R（靜止風險）分析
- S（交易穩定度）分析
- V（終生價值）分析
- 27 種客戶類型分類
- 策略建議對應

8.2 R（靜止風險）計算

基礎: 基於 Recency (`r_value`)

```
# 計算百分位數
r_percentile_80 = quantile(r_value, 0.8, na.rm = TRUE)
r_percentile_20 = quantile(r_value, 0.2, na.rm = TRUE)

# 分級
r_level = case_when(
  r_value >= r_percentile_80 ~ "高",      # P80 以上 = 高靜止風險
  r_value >= r_percentile_20 ~ "中",      # P20-P80
  TRUE ~ "低"                            # P20 以下 = 低靜止風險
)

# 標籤
tag_032_dormancy_risk = case_when(
  r_level == "高" ~ "高靜止戶",
```

```

    r_level == "中" ~ "中靜止戶",
    TRUE ~ "低靜止戶"
)

```

邏輯解釋：

- R值 = 最後購買距今天數
- R值越大 → 客戶越久沒購買 → 靜止風險越高
- 使用百分位數（相對排名）而非絕對天數

示例：

假設 r_value 分佈：
 P20 = 10 天, P80 = 60 天

客戶 A: r_value = 70 → r_level = "高" → 高靜止戶

客戶 B: r_value = 30 → r_level = "中" → 中靜止戶

客戶 C: r_value = 5 → r_level = "低" → 低靜止戶

8.3 S（交易穩定度）計算

策略 A：使用交易次數 (ni) 作為穩定度代理

```

# 使用 ni 作為穩定度指標（更高 = 更穩定）
stability_metric = ni

# 計算百分位數
s_percentile_80 = quantile(stability_metric, 0.8, na.rm = TRUE)
s_percentile_20 = quantile(stability_metric, 0.2, na.rm = TRUE)

# 分級（ni 越高越穩定）
s_level = case_when(
  stability_metric >= s_percentile_80 ~ "高",    # P80 以上 = 高穩定
  stability_metric >= s_percentile_20 ~ "中",    # P20-P80
  TRUE ~ "低"                                   # P20 以下 = 低穩定
)

# 標籤
tag_033_transaction_stability = case_when(
  s_level == "高" ~ "高穩定",
  s_level == "中" ~ "中穩定",
  TRUE ~ "低穩定"
)

```

策略 B：使用 IPT 變異係數（如果可用）

```
# 計算變異係數 (CV) = 標準差 / 平均值
# 需要每位客戶的多次 IPT 數據
stability_cv = ipt_sd / ipt_mean

# 計算百分位數
s_percentile_20 = quantile(stability_cv, 0.2, na.rm = TRUE)
s_percentile_80 = quantile(stability_cv, 0.8, na.rm = TRUE)

# 分級 (CV 越小越穩定，所以反轉)
s_level = case_when(
  stability_cv <= s_percentile_20 ~ "高", # P20 以下 = 高穩定 (購買規律)
  stability_cv <= s_percentile_80 ~ "中", # P20-P80
  TRUE ~ "低" # P80 以上 = 低穩定 (購買不規律)
)
```

兩種策略比較：

策略	指標	優點	缺點	適用場景
策略 A	ni (交易次數)	簡單，數據容易獲得	不反映購買規律性	數據不足時
策略 B	CV (變異係數)	真實反映購買穩定性	需要時間序列數據	數據充足時

當前實作: 使用策略 A (ni)

示例 (策略 A)：

```
假設 ni 分佈：
P20 = 2 次, P80 = 10 次

客戶 A: ni = 15 → s_level = "高" → 高穩定 (交易次數多)
客戶 B: ni = 5  → s_level = "中" → 中穩定
客戶 C: ni = 1  → s_level = "低" → 低穩定 (交易次數少)
```

示例 (策略 B)：

```
假設 CV 分佈：
P20 = 0.2, P80 = 0.8

客戶 A: CV = 0.15 → s_level = "高" → 高穩定 (購買規律)
客戶 B: CV = 0.5  → s_level = "中" → 中穩定
客戶 C: CV = 0.9  → s_level = "低" → 低穩定 (購買不規律)
```

8.4 V (終生價值) 計算

基礎: 簡化客戶終生價值 (CLV) 計算

```

# CLV 簡化公式：年度化價值
# CLV = m_value × (f_value / 365) × 365 = m_value × f_value
clv = m_value * f_value

# 計算百分位數
v_percentile_80 = quantile(clv, 0.8, na.rm = TRUE)
v_percentile_20 = quantile(clv, 0.2, na.rm = TRUE)

# 分級
v_level = case_when(
  clv >= v_percentile_80 ~ "高",      # P80 以上 = 高終生價值
  clv >= v_percentile_20 ~ "中",      # P20-P80
  TRUE ~ "低"                        # P20 以下 = 低終生價值
)

# 標籤
tag_034_customer_lifetime_value = case_when(
  v_level == "高" ~ "高價值",
  v_level == "中" ~ "中價值",
  TRUE ~ "低價值"
)

```

CLV 計算邏輯：

- 簡化公式: $CLV = m_value \times f_value$
- 解釋: 平均客單價 × 購買頻率 = 年度化價值
- 假設: 未來購買行為與過去相似

進階 CLV 計算（可選）：

```

# 更精確的 CLV 公式（如果有更多數據）
CLV = (平均客單價 × 購買頻率 × 客戶生命週期) - 獲客成本

# 或使用生存分析預測客戶生命週期

```

示例：

```

客戶 A: m_value=1000, f_value=2 (次/月)
→ clv = 1000 × 2 = 2000
→ 假設 P80 = 1500
→ v_level = "高" → 高價值

客戶 B: m_value=500, f_value=1
→ clv = 500 × 1 = 500
→ v_level = "低" → 低價值

```

8.5 27 種客戶類型分類

邏輯: $R \times S \times V$ 的所有組合 ($3 \times 3 \times 3 = 27$)

```
# 生成組合 key
rsv_key = paste0(r_level, s_level, v_level) # 例如 "低高高"

# 27 種客戶類型對應 (部分示例)
customer_type = case_when(
  rsv_key == "低高高" ~ "金鑽客", # R低=活躍, S高=穩定, V高=高價值
  rsv_key == "低高中" ~ "成長型忠誠客",
  rsv_key == "低高低" ~ "潛力忠誠客",
  rsv_key == "低中高" ~ "波動高值客",
  rsv_key == "中高高" ~ "預警VIP", # R中=開始不活躍
  rsv_key == "高高高" ~ "流失風險VIP", # R高=很不活躍
  rsv_key == "高低高" ~ "流失高值客",
  rsv_key == "高低低" ~ "沉睡客",
  # ... 共 27 種
  TRUE ~ "一般客群"
)
```

27 種類型矩陣：

R層級 = 低 (活躍)			
S高 V高 金鑽客	S中 V高 波動高值客	S低 V高 不穩高值客	
S高 V中 成長忠誠客	S中 V中 穩定中值客	S低 V中 波動中值客	
S高 V低 潛力忠誠客	S中 V低 標準客戶	S低 V低 不穩低值客	
R層級 = 中 (預警)			
S高 V高 預警VIP	S中 V高 搖擺高值客	S低 V高 風險高值客	
S高 V中 預警忠誠客	S中 V中 標準預警客	S低 V中 不穩預警客	
S高 V低 潛力預警客	S中 V低 一般預警客	S低 V低 低值預警客	
R層級 = 高 (流失)			
S高 V高 流失風險VIP	S中 V高 流失高值客	S低 V高 已流失高值	

S高 V中	S中 V中	S低 V中
流失中值客	沉睡中值客	已流失中值
S高 V低	S中 V低	S低 V低
流失低值客	沉睡客	可放棄客戶

8.6 策略建議對應

核心類型策略示例：

RSV組合	客戶類型	策略重點	行動方案
低高高	金鑽客	VIP體驗+品牌共創	專屬客服、新品搶先體驗、會員大使
低高中	成長型忠誠客	升級誘因	搭售組合、滿額升級、會員積分任務
中高高	預警VIP	早期挽回	回購提醒、VIP喚醒禮、定向廣告
高高高	流失風險VIP	挽回行銷	再行銷廣告、專屬優惠、客服致電
高低低	沉睡客	低成本回流	廣告再曝光、再註冊誘因

8.7 生成的標籤

標籤編號	標籤名稱	值域	計算條件	說明
tag_032	dormancy_risk	高/中/低靜止戶	所有客戶	基於 R值
tag_033	transaction_stability	高/中/低穩定	所有客戶	基於 ni 或 CV
tag_034	customer_lifetime_value	高/中/低價值	所有客戶	基於 CLV

8.8 UI 輸出

R/S/V 分佈：

- 靜止風險分佈圓餅圖
- 交易穩定度分佈圓餅圖
- 終生價值分佈圓餅圖

27 種客戶類型矩陣：

- 3D 矩陣視覺化（或多層次表格）
- 每個格子顯示：客戶類型、客戶數、策略
- 可點擊查看詳細客戶名單

CSV 匯出：

- 包含 customer_id, email, r_level, s_level, v_level
- 客戶類型、策略建議、預估 CLV
- 用於廣告投放或 CRM 系統

9. 模組6：生命週期預測模組

檔案: `modules/module_lifecycle_prediction.R` 主要函數:
`lifecyclePredictionModuleServer()`

9.1 模組功能

預測客戶下次購買行為：

- 下次購買金額預測
- 下次購買日期預測
- 預測信心度評估

9.2 下次購買金額預測

計算邏輯：

```
tag_030_next_purchase_amount = tag_004_avg_order_value
```

假設：

- 過去平均購買金額 = 未來購買金額
- 客戶購買行為穩定

適用性：

- ☒ 對穩定客戶預測較準
- ☐ 對新客或波動客戶預測不可靠

示例：

```
客戶 A: tag_004_avg_order_value = 1200 元  
→ tag_030_next_purchase_amount = 1200 元
```

```
客戶 B: tag_004_avg_order_value = 500 元  
→ tag_030_next_purchase_amount = 500 元
```

9.3 下次購買日期預測

計算邏輯：

```
tag_031_next_purchase_date = as.Date(Sys.time()) +  
tag_001_avg_purchase_cycle
```

公式：

預測購買日期 = 今天 + 平均購買週期

假設：

- 客戶將在平均購買週期後再次購買
- 購買週期穩定

示例：

今天 = 2025-10-25

客戶 A: tag_001_avg_purchase_cycle = 30 天
→ tag_031_next_purchase_date = 2025-11-24

客戶 B: tag_001_avg_purchase_cycle = 60 天
→ tag_031_next_purchase_date = 2025-12-24

9.4 預測信心度分類

邏輯：

confidence = case_when(
 ni >= 4 ~ "高 (≥4次) ",
 ni >= 2 ~ "中 (2-3次) ",
 TRUE ~ "低 (1次) "
)

交易次數充足，預測可靠
交易次數中等，預測中等可靠
交易次數不足，預測不可靠

信心度等級：

等級	條件	說明	建議使用
高	ni ≥ 4	有足夠歷史數據，預測可靠	✔ 可直接使用於行銷
中	2 ≤ ni < 4	數據中等，預測參考價值中	⚠ 謹慎使用，結合其他指標
低	ni = 1	數據不足，預測不可靠	✘ 不建議使用

9.5 生成的標籤

標籤編號	標籤名稱	計算公式	計算條件	說明
tag_030	next_purchase_amount	tag_004_avg_order_value	所有客戶	預測購買金額
tag_031	next_purchase_date	今天 + tag_001_avg_purchase_cycle	所有客戶	預測購買日期

9.6 進階預測方法（未實作，可選）

方法 A：加權移動平均

```
# 給近期交易更高權重
recent_weights = exp(-seq(1, ni) / ni)
tag_030_next_purchase_amount = weighted.mean(transaction_amounts,
recent_weights)
```

方法 B：趨勢調整

```
# 考慮金額趨勢（上升/下降）
trend = lm(amount ~ time)$coefficients[2]
tag_030_next_purchase_amount = last_amount + trend * avg_ipt
```

方法 C：機器學習預測

```
# 使用歷史數據訓練模型
model <- train(next_amount ~ r_value + f_value + m_value + ..., data =
train_data)
tag_030_next_purchase_amount = predict(model, newdata = customer_data)
```

9.7 UI 輸出

預測結果：

- 下次購買金額預測（平均值）
- 下次購買日期預測（平均值）
- 信心度分佈（高/中/低客戶數）

視覺化：

- 預測金額分佈直方圖
- 預測日期時間線
- 信心度圓餅圖

匯出：

- CSV 包含預測結果與信心度
- 用於行銷活動規劃

10. 客戶標籤計算工具

檔案: `utils/calculate_customer_tags.R`

10.1 工具架構

```
# 主函數：計算所有標籤
calculate_all_customer_tags(customer_data)
├─ calculate_base_value_tags()      # 基礎價值標籤 (tag_001-004)
├─ calculate_rfm_tags()            # RFM 標籤 (tag_009-013)
│   └─ calculate_rfm_scores()      # RFM 分數計算
├─ calculate_status_tags()         # 狀態標籤 (tag_017-019)
└─ calculate_prediction_tags()     # 預測標籤 (tag_030-031)
```

10.2 標籤計算流程

步驟 1：基礎價值標籤

```
calculate_base_value_tags <- function(customer_data) {
  customer_data %>%
    mutate(
      tag_001_avg_purchase_cycle = ipt_value,
      tag_003_historical_total_value = m_value,
      tag_004_avg_order_value = m_value / ni
    )
}
```

步驟 2：RFM 標籤

```
calculate_rfm_tags <- function(customer_data) {
  # 先計算 RFM 分數 (只對 ni >= 4)
  customer_data <- calculate_rfm_scores(customer_data)

  customer_data %>%
    mutate(
      tag_009_rfm_r = r_value,
      tag_010_rfm_f = f_value,
      tag_011_rfm_m = m_value,
      # tag_012_rfm_score 已在 calculate_rfm_scores() 中計算
      tag_013_value_segment = value_level
    )
}
```

步驟 3：狀態標籤

```
calculate_status_tags <- function(customer_data) {
  customer_data %>%
    mutate(
      tag_017_lifecycle_stage = case_when(
        lifecycle_stage == "newbie" ~ "新客",
        lifecycle_stage == "active" ~ "主力客",
        lifecycle_stage == "sleepy" ~ "瞌睡客",
      )
    )
}
```

```

    lifecycle_stage == "half_sleepy" ~ "半睡客",
    lifecycle_stage == "dormant" ~ "沉睡客",
    TRUE ~ "未知"
  ),

  tag_018_churn_risk = case_when(
    ni == 1 ~ "新客 (無法評估)",
    ni < 4 ~ if_else(r_value > 30, "中風險", "低風險"),
    r_value > ipt_value * 2 ~ "高風險",
    r_value > ipt_value * 1.5 ~ "中風險",
    TRUE ~ "低風險"
  ),

  tag_019_days_to_churn = case_when(
    ni == 1 ~ NA_real_,
    ni < 4 ~ NA_real_,
    TRUE ~ pmax(0, ipt_value * 2 - r_value)
  )
)
}

```

步驟 4：預測標籤

```

calculate_prediction_tags <- function(customer_data) {
  customer_data %>%
    mutate(
      tag_030_next_purchase_amount = tag_004_avg_order_value,
      tag_031_next_purchase_date = as.Date(Sys.time()) +
tag_001_avg_purchase_cycle
    )
}

```

步驟 5：整合

```

calculate_all_customer_tags <- function(customer_data) {
  customer_data %>%
    calculate_base_value_tags() %>%
    calculate_rfm_tags() %>%
    calculate_status_tags() %>%
    calculate_prediction_tags()
}

```

10.3 標籤依賴關係

```

DNA 分析輸出
├─ r_value
│   └─ tag_009_rfm_r

```



11. 所有標籤彙總表

11.1 基礎價值標籤 (tag_001 - tag_004)

標籤	名稱	計算公式	單位	條件	模組
tag_001	平均購買週期	ipt_value	天	所有	基礎價值
tag_003	歷史總價值	m_value	金額	所有	基礎價值
tag_004	平均客單價	m_value / ni	金額	所有	基礎價值

11.2 RFM 分析標籤 (tag_009 - tag_013)

標籤	名稱	計算公式	範圍	條件	模組
tag_009	RFM_R	r_value	天數	所有	RFM
tag_010	RFM_F	f_value	次/月	所有	RFM
tag_011	RFM_M	m_value	金額	所有	RFM
tag_012	RFM總分	r_score + f_score + m_score	3-15	ni ≥ 4	RFM
tag_013	價值分群	value_level	高/中/低	所有	RFM

11.3 客戶狀態標籤 (tag_017 - tag_019)

標籤	名稱	值域	條件	模組
----	----	----	----	----

標籤	名稱	值域	條件	模組
tag_017	生命週期階段	新客/主力客/睡眠客/半睡客/沉睡客	所有	狀態
tag_018	流失風險	高風險/中風險/低風險/新客（無法評估）	所有	狀態
tag_019	預估流失天數	天數或 NA	ni ≥ 4	狀態

11.4 預測標籤 (tag_030 - tag_031)

標籤	名稱	計算公式	單位	條件	模組
tag_030	下次購買金額	tag_004	金額	所有	預測
tag_031	下次購買日期	今天 + tag_001	日期	所有	預測

11.5 R/S/V 矩陣標籤 (tag_032 - tag_034)

標籤	名稱	值域	條件	模組
tag_032	靜止風險	高/中/低靜止戶	所有	RSV
tag_033	交易穩定度	高/中/低穩定	所有	RSV
tag_034	終生價值	高/中/低價值	所有	RSV

11.6 標籤計算條件總覽

計算條件矩陣：

標籤類型	所有客戶	ni ≥ 4	ni < 4	ni = 1
基礎價值 (001-004)	✓			
RFM 原始值 (009-011)	✓			
RFM 分數 (012)			✓	NA
價值分群 (013)	✓			NA
生命週期 (017)	✓			
流失風險 (018)	✓			簡化邏輯
流失天數 (019)			✓	NA
預測 (030-031)	✓			低信心
RSV (032-034)	✓			極低信心

12. 分群閾值與百分位數

12.1 百分位數統一標準

全域規則：80/20 法則（帕累托原則）

分群維度	低	中	高	說明
------	---	---	---	----

分群維度	低	中	高	說明
活躍度 (CAI)	<0.2	0.2-0.8	≥0.8	ECDF 值
價值 (M)	<P20	P20-P80	≥P80	Quantile
R值	<P20	P20-P80	≥P80	Quantile
F值	<P20	P20-P80	≥P80	Quantile
M值	<P20	P20-P80	≥P80	Quantile
靜止風險 (R)	<P20	P20-P80	≥P80	Quantile
穩定度 (S)	<P20	P20-P80	≥P80	Quantile
終生價值 (V)	<P20	P20-P80	≥P80	Quantile

P20/P80 解釋：

- **P20 (20th percentile):** 20% 的客戶在此閾值以下
- **P80 (80th percentile):** 80% 的客戶在此閾值以下
- **80/20 法則:** 20% 的客戶貢獻 80% 的價值

12.2 RFM 5分制百分位數

用於更細緻的 RFM 分數計算

分數	R值範圍	F值範圍	M值範圍
5 分	≤P20	≥P80	≥P80
4 分	P20-P40	P60-P80	P60-P80
3 分	P40-P60	P40-P60	P40-P60
2 分	P60-P80	P20-P40	P20-P40
1 分	>P80	<P20	<P20

注意：

- R值：越小越好（最近購買 = 高分）
- F值：越大越好（高頻率 = 高分）
- M值：越大越好（高價值 = 高分）

12.3 硬編碼閾值

生命週期階段（R值天數）

階段	R值閾值	說明	可配置
主力客	$R \leq 7$ 天	固定值	△ 建議改為參數
瞌睡客	$7 < R \leq 14$ 天	固定值	△ 建議改為參數

階段	R值閾值	說明	可配置
半睡客	$14 < R \leq 21$ 天	固定值	△ 建議改為參數
沉睡客	$R > 21$ 天	固定值	△ 建議改為參數

流失風險（IPT 倍數）

風險等級	閾值	說明	可配置
高風險	$R > 2 \times \text{IPT}$	固定倍數	△ 建議改為參數
中風險	$1.5 \times \text{IPT} < R \leq 2 \times \text{IPT}$	固定倍數	△ 建議改為參數
低風險	$R \leq 1.5 \times \text{IPT}$	固定倍數	△ 建議改為參數

交易次數閾值

用途	閾值	說明	可配置
活躍度完整計算	$n_i \geq 4$	統計學建議	✓ 符合規劃
RFM 分數計算	$n_i \geq 4$	同上	✓ 符合規劃
預估流失天數	$n_i \geq 4$	數據可靠性	✓ 符合規劃
最小樣本數	$n \geq 30$	統計學標準	△ 建議改為參數
建議樣本數	$n \geq 100$	百分位穩定性	△ 建議改為參數

12.4 閾值彙總表

閾值彙總表				
類型	閾值	類型	來源	
活躍度（CAI）	0.2/0.8	ECDF	動態	
價值（M）	P20/P80	Quantile	動態	
R分數	P20/40/60/80	Quantile	動態	
F分數	P20/40/60/80	Quantile	動態	
M分數	P20/40/60/80	Quantile	動態	
R/S/V	P20/P80	Quantile	動態	
主力客	$R \leq 7$ 天	固定值	硬編碼	
瞌睡客	$7 < R \leq 14$ 天	固定值	硬編碼	
半睡客	$14 < R \leq 21$ 天	固定值	硬編碼	
沉睡客	$R > 21$ 天	固定值	硬編碼	
高風險	$R > 2 \times \text{IPT}$	倍數	硬編碼	
中風險	$1.5 \times \text{IPT} < R \leq 2 \times \text{IPT}$	倍數	硬編碼	
低風險	$R \leq 1.5 \times \text{IPT}$	倍數	硬編碼	
ni閾值	≥ 4	固定值	規劃	

樣本數下限	≥30	固定值	統計
樣本數建議	≥100	固定值	統計

13. 邏輯一致性檢查

13.1 相同指標計算一致性

✅ 一致的指標

指標	DNA 九宮格	基礎價值	RFM	狀態	RSV	預測	結論
r_value	計算	使用	使用	使用	使用	-	✅ 一致
f_value	計算	使用	使用	-	使用	-	✅ 一致
m_value	計算	使用	使用	-	使用	使用	✅ 一致
ipt_value	計算	使用	-	使用	-	使用	✅ 一致
ni	計算	使用	使用	使用	使用	使用	✅ 一致
lifecycle_stage	計算	-	-	語言轉換	-	-	✅ 一致
activity_level	計算	-	-	-	-	-	✅ 唯一

⚠ 有差異的指標

1. tag_013_value_segment 定義差異

模組	計算方式	基礎指標	說明
DNA 九宮格	quantile(m_value, 0.2/0.8)	m_value	過去價值（總消費）
RFM 分析	使用 value_level	value_level	直接引用 DNA 結果
RSV 矩陣	quantile(clv, 0.2/0.8)	m_value × f_value	終生價值（CLV）

結論：

- DNA 和 RFM 中的 value_segment 一致（基於 m_value）
- RSV 中的 v_level 不同（基於 CLV = m_value × f_value）
- ✅ 這是設計上的差異，不是錯誤
- 建議：在 RSV 中使用不同的標籤名稱避免混淆

2. 新客定義表述差異 (✅ GAP-001 v3 已統一 2025-10-26)

模組	定義 (v3 更新後)	代碼
DNA 九宮格	ni == 1	Line 391-397
狀態模組	ni == 1	Line 181

歷史差異 (已解決)：

- v1: DNA 九宮格使用 ~~ni == 1 & customer_age_days ≤ avg_ipt~~ (已廢棄)
- v2: DNA 九宮格使用 ~~ni == 1 & customer_age_days ≤ 60~~ (已廢棄)
- ☒ v3: 所有模組統一使用 **ni == 1** (最終版本)

結論：

- ☒ v3 已統一：所有模組使用相同定義 (ni == 1)
- 簡化理由：業務清晰度、100% 覆蓋率、符合實務需求

13.2 百分位數計算一致性

檢查項目：所有使用百分位數的地方是否統一

分群維度	閾值類型	P20	P80	計算範圍	一致性
活躍度 (CAI)	ECDF	0.2	0.8	ni ≥ 4	☒
價值 (M)	Quantile	P20	P80	全體	☒
R/F/M (RFM)	Quantile	P20,P40,P60,P80	-	ni ≥ 4	☒
R/S/V	Quantile	P20	P80	全體	☒

結論：☒ 所有百分位數計算統一使用 P20/P80 (或細分為 P20/P40/P60/P80)

13.3 交易次數篩選一致性

檢查項目：ni < 4 的處理是否一致

計算內容	ni = 1	ni = 2-3	ni ≥ 4	一致性
activity_level	NA	r_value降級	CAI	☒
RFM 分數	NA	NA	計算	☒
流失風險	無法評估	簡化邏輯	完整邏輯	☒
流失天數	NA	NA	計算	☒
預測	低信心	中信心	高信心	☒

結論：☒ 所有模組統一正確處理 ni < 4 的情況

13.4 硬編碼閾值檢查

潛在問題：固定閾值可能不適用所有行業

閾值	當前值	問題	建議
R ≤ 7 天	固定	不同行業購買週期差異大	改為 $R \leq avg_ipt \times 0.25$
R ≤ 14 天	固定	同上	改為 $R \leq avg_ipt \times 0.5$

閾值	當前值	問題	建議
$R \leq 21 \text{ 天}$	固定	同上	改為 $R \leq \text{avg_ipt} \times 0.75$
$R > 2 \times \text{IPT}$	固定倍數	合理，但可配置化	改為參數 <code>churn_multiplier = 2</code>
$R > 1.5 \times \text{IPT}$	固定倍數	同上	改為參數 <code>warning_multiplier = 1.5</code>

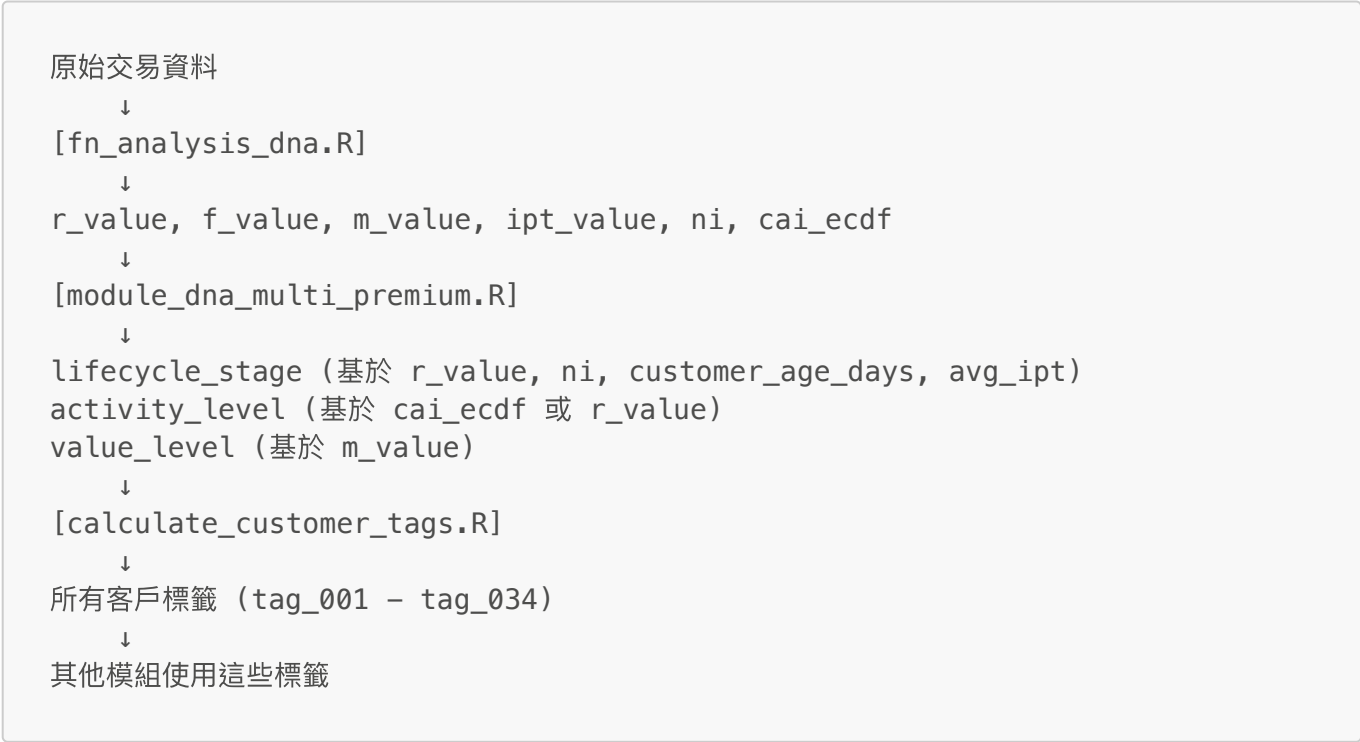
建議改進方案：

```
# 當前（硬編碼）
lifecycle_stage = case_when(
  r_value <= 7 ~ "active",
  r_value <= 14 ~ "sleepy",
  r_value <= 21 ~ "half_sleepy",
  TRUE ~ "dormant"
)

# 建議（參數化）
lifecycle_stage = case_when(
  r_value <= avg_ipt * 0.25 ~ "active",      # 1/4 平均週期
  r_value <= avg_ipt * 0.50 ~ "sleepy",     # 1/2 平均週期
  r_value <= avg_ipt * 0.75 ~ "half_sleepy",# 3/4 平均週期
  TRUE ~ "dormant"
)
```

13.5 數據依賴鏈檢查

完整依賴鏈：



依賴鏈完整性：✅ 無循環依賴，數據流向清晰

14. 已知問題與建議改進

14.1 已知問題

問題 1：硬編碼閾值不適應所有行業

問題描述：

- 生命週期階段使用固定 R 值閾值 (7/14/21 天)
- 不同行業購買週期差異大 (快消品 vs 耐用品)

影響範圍：

- 生命週期階段分類可能不準確
- 流失風險評估可能失效

建議改進：

```
# 使用平均購買週期的百分比
lifecycle_stage = case_when(
  r_value <= avg_ipt * 0.25 ~ "active",
  r_value <= avg_ipt * 0.50 ~ "sleepy",
  r_value <= avg_ipt * 0.75 ~ "half_sleepy",
  TRUE ~ "dormant"
)
```

優先級：● 高

問題 2：tag_013_value_segment 定義重複

問題描述：

- DNA 九宮格：基於 m_value
- RSV 矩陣：基於 CLV (m_value × f_value)
- 使用相同的標籤名稱但定義不同

影響範圍：

- 可能造成用戶混淆
- 不同模組顯示不一致

建議改進：

```
# RSV 矩陣中使用不同名稱
tag_034_customer_lifetime_value # 已存在
v_level = "高價值" # 基於 CLV

# 保持 tag_013 只在 RFM 中使用 (基於 m_value)
```

優先級：🟡 中

問題 3：小樣本百分位數不穩定

問題描述：

- 當樣本數 < 30 時，百分位數不可靠
- 極端值會嚴重影響分群

影響範圍：

- 所有基於百分位數的分群
- 客戶數量少的業務

建議改進：

```
# 設置最小樣本數要求
if (n_customers < 30) {
  showNotification("⚠ 樣本數不足，分群結果可能不可靠", type = "warning")
}

if (n_customers < 100) {
  # 使用固定閾值或合併分組（高/低兩級）
  value_level = case_when(
    m_value >= median(m_value) ~ "高",
    TRUE ~ "低"
  )
}
```

優先級：🟡 中

問題 4：CAI 計算條件不清晰

問題描述：

- CAI 只在 `ni >= 4` 且 `times != 1` 時計算
- 用戶可能不理解為什麼某些客戶沒有 CAI

影響範圍：

- 活躍度計算透明度
- 用戶理解困難

建議改進：


```
# 在 UI 中顯示說明
output$cai_explanation <- renderUI({
  tags$div(
    class = "alert alert-info",
    icon("info-circle"),
    " CAI (客戶活躍度指數) 只針對交易次數 ≥ 4 的客戶計算。",
    "交易次數不足的客戶使用最近購買天數 (R值) 作為活躍度代理指標。"
  )
})
```

優先級：● 低

14.2 建議改進清單

改進 1：參數化配置系統

目標：將所有硬編碼閾值改為可配置參數

實作方案：

```
# 建立配置檔案：config/thresholds.yaml
lifecycle:
  active_multiplier: 0.25    #  $R \leq \text{avg\_ipt} \times 0.25$ 
  sleepy_multiplier: 0.50   #  $R \leq \text{avg\_ipt} \times 0.50$ 
  half_sleepy_multiplier: 0.75

churn_risk:
  high_risk_multiplier: 2.0  #  $R > \text{avg\_ipt} \times 2.0$ 
  medium_risk_multiplier: 1.5 #  $R > \text{avg\_ipt} \times 1.5$ 

sampling:
  min_sample_size: 30
  recommended_sample_size: 100

activity:
  min_transactions: 4 # ni 閾值

# 在代碼中讀取
config <- yaml::read_yaml("config/thresholds.yaml")
```

優先級：● 高

改進 2：增加信心度指標

目標：為所有預測和分群結果增加信心度評估

實作方案：

```
# 計算信心度
confidence_score = case_when(
  ni >= 10 ~ 0.95, # 高信心
  ni >= 4 ~ 0.80, # 中高信心
  ni >= 2 ~ 0.60, # 中信心
  TRUE ~ 0.30    # 低信心
)

# 在 UI 中顯示
output$confidence_badge <- renderUI({
  if (confidence_score >= 0.8) {
    tags$span(class = "badge badge-success", "高信心")
  } else if (confidence_score >= 0.6) {
    tags$span(class = "badge badge-warning", "中信心")
  } else {
    tags$span(class = "badge badge-danger", "低信心")
  }
})
```

優先級：🟡 中

改進 3：進階預測模型

目標：使用機器學習改善預測準確度

實作方案：

```
# 訓練預測模型
library(caret)

# 特徵工程
features <- customer_data %>%
  select(r_value, f_value, m_value, ipt_value, ni,
         customer_age_days, lifecycle_stage)

# 訓練模型（預測下次購買金額）
model_amount <- train(
  next_amount ~ .,
  data = train_data,
  method = "rf", # Random Forest
  trControl = trainControl(method = "cv", number = 5)
)

# 預測
tag_030_next_purchase_amount = predict(model_amount, newdata =
customer_data)
```

優先級：🟢 低（進階功能）

改進 4：異常值處理

目標：識別並處理極端值，避免影響百分位數計算

實作方案：

```
# Winsorize (極端值修剪)
winsorize <- function(x, probs = c(0.01, 0.99)) {
  quantiles <- quantile(x, probs, na.rm = TRUE)
  x[x < quantiles[1]] <- quantiles[1]
  x[x > quantiles[2]] <- quantiles[2]
  return(x)
}

# 應用到關鍵指標
customer_data <- customer_data %>%
  mutate(
    m_value_winsorized = winsorize(m_value),
    r_value_winsorized = winsorize(r_value)
  )

# 使用修剪後的值計算百分位數
```

優先級：🟡 中

改進 5：自動化測試

目標：建立單元測試確保邏輯一致性

實作方案：

```
# tests/test_logic_consistency.R
library(testthat)

test_that("新客定義一致", {
  # 測試案例
  customer <- data.frame(
    ni = 1,
    customer_age_days = 15,
    avg_ipt = 30
  )

  # 預期結果
  expect_equal(
    calculate_lifecycle_stage(customer)$lifecycle_stage,
    "newbie"
  )
})
```

```
test_that("活躍度計算一致", {
  # ni >= 4 時應使用 CAI
  customer <- data.frame(ni = 5, cai_ecdf = 0.85)
  expect_equal(
    calculate_activity_level(customer)$activity_level,
    "高"
  )

  # ni < 4 時應使用 r_value
  customer <- data.frame(ni = 3, r_value = 5, cai_ecdf = NA)
  expect_equal(
    calculate_activity_level(customer)$activity_level,
    "高" # 假設 r_value=5 在 P20 以下
  )
})
```

優先級：● 高

14.3 改進優先級總覽

改進項目	優先級	預估工時	影響範圍
參數化配置系統	● 高	4 小時	所有模組
自動化測試	● 高	6 小時	全系統
異常值處理	● 中	3 小時	分群邏輯
信心度指標	● 中	2 小時	預測模組
tag_013 重複定義	● 中	1 小時	RFM/RSV
CAI 說明	● 低	0.5 小時	UI
進階預測模型	● 低	8 小時	預測模組

📌 文檔維護說明

更新頻率

- 每次邏輯變更後立即更新
- 每月定期審查一次
- 重大版本發布前完整審核

更新負責人

- 開發團隊：修改代碼後同步更新文檔
- 文檔管理員：定期審查一致性

版本記錄

- **v2.0** (2025-10-25): 完整重寫，包含所有模組詳細邏輯
 - **v1.0** (2025-10-20): 初始版本
-

文檔結束

如有任何問題或需要澄清的地方，請聯繫開發團隊。